

ร่างขอบเขตของงาน

(Terms of Reference: TOR)

โครงการพัฒนาระบบ LINE Auto Reports

ระบบบริหารรายงานอัตโนมัติและจัดส่งรายงานไปยัง LINE Group ตามเงื่อนไขและเวลาที่กำหนด

รายการ	รายละเอียด
หน่วยงานเจ้าของโครงการ	[ระบุหน่วยงาน]
ชื่อโครงการ	การพัฒนาระบบ LINE Auto Reports
ประเภทระบบ	Web Application / Backend Service / LINE Messaging Integration
เทคโนโลยีหลักที่เสนอ	Next.js, Node.js, PostgreSQL, LINE Messaging API, Scheduler
ระยะเวลาดำเนินการที่เสนอ	120 วันปฏิทินนับถัดจากวันที่ลงนามในสัญญา
งบประมาณ	[ระบุวงเงินตามแผนจัดซื้อจัดจ้าง]
ฉบับ	Draft 1.0
วันที่จัดทำ	14 สิงหาคม 2569

หลักการสำคัญของโครงการ

เปลี่ยนการบริหารรายงานจากรูปแบบที่กำหนดไว้ใน PHP/ไฟล์ Cron แบบตายตัว ไปสู่ระบบที่ผู้ดูแลสามารถกำหนด “กลุ่ม LINE → รายงาน → กลุ่มเป้าหมาย → ตารางเวลา” ผ่าน Back Office โดยไม่ต้องแก้ Source Code หรือสร้าง Cron Job ใหม่ในทุกครั้งที่เพิ่มรายงานหรือเปลี่ยนกลุ่มเป้าหมาย

สารบัญและโครงสร้างเอกสาร

1. หลักการและเหตุผล
2. วัตถุประสงค์
3. Business Value และประโยชน์ที่คาดว่าจะได้รับ
4. ขอบเขตของระบบ
5. แนวคิดและสถาปัตยกรรมระบบ
6. Functional Requirements
7. Non-Functional Requirements
8. โครงสร้างข้อมูลและฐานข้อมูล
9. การเชื่อมต่อ LINE Messaging API
10. Scheduler และ Report Execution Engine
11. Back Office และสิทธิ์ผู้ใช้งาน
12. ความมั่นคงปลอดภัยและการคุ้มครองข้อมูล
13. Logging, Monitoring และ Audit Trail
14. API และการเชื่อมต่อกับระบบอื่น
15. การย้ายจากระบบ PHP/Cron เดิม
16. ผลส่งมอบ (Deliverables)
17. แผนดำเนินงานและ Timeline
18. เกณฑ์การทดสอบและตรวจรับ
19. การรับประกันและบำรุงรักษา
20. ข้อกำหนดด้านผู้พัฒนา/ผู้รับจ้าง
21. เงื่อนไขสำคัญและข้อพึงระวัง
22. สรุปภาพรวมโครงการ

1. หลักการและเหตุผล

หน่วยงานมีการจัดทำรายงานจากข้อมูลของระบบสารสนเทศหลายประเภท เช่น รายงานการให้บริการ รายงานการเข้าใช้งานระบบ รายงานนักศึกษา รายงานด้านการเงิน รายงานความก้าวหน้าโครงการ และรายงานเพื่อประกอบการบริหาร โดยปัจจุบันการส่งรายงานไปยัง LINE Group สามารถดำเนินการผ่านโปรแกรม PHP และ Cron Job ซึ่งเหมาะสมกับการเริ่มต้นใช้งาน แต่เมื่อจำนวนรายงานและกลุ่มผู้รับเพิ่มขึ้น จะทำให้การบริหารจัดการ Source Code, Cron Job, เวลาในการส่ง และกลุ่มเป้าหมายมีความซับซ้อนมากขึ้น

โครงการ LINE Auto Reports มีวัตถุประสงค์เพื่อยกระดับรูปแบบดังกล่าวเป็นระบบกลางที่มีการจัดการแบบ Configuration-driven โดยแยกส่วนการกำหนดรายงาน กลุ่ม LINE กลุ่มเป้าหมาย และตารางเวลาออกจาก Source Code ทำให้สามารถเพิ่มหรือเปลี่ยนแปลงการจัดส่งรายงานได้จาก Back Office

ระบบจะเชื่อมต่อกับ LINE Official Account ผ่าน LINE Messaging API โดยเมื่อ LINE Official Account ถูกเชิญเข้า Group ระบบจะรับ Webhook Event ประเภท join เพื่อรับ groupId และสามารถเรียก Group Summary API เพื่อดึงชื่อกลุ่มและข้อมูลประกอบ จากนั้นจัดเก็บข้อมูลกลุ่มในฐานข้อมูลเพื่อใช้เป็นปลายทางของรายงานอัตโนมัติ

หลักการออกแบบ

Database เป็นแหล่งกำหนด Configuration ของการจัดส่งรายงาน ส่วน Report Engine รับผิดชอบการประมวลผลข้อมูล การสร้างข้อความ/ Flex Message และการส่งผ่าน LINE Messaging API ขณะที่ Scheduler ทำหน้าที่ตรวจสอบงานที่ถึงเวลาและเรียก Report Engine

2. วัตถุประสงค์

- พัฒนาระบบกลางสำหรับบริหาร LINE Group และจัดเก็บ groupId อย่างเป็นระบบ
- พัฒนาระบบจัดการนิยามรายงาน (Report Definition) ที่สามารถนำกลับมาใช้ซ้ำได้
- กำหนดกลุ่มเป้าหมายของแต่ละรายงานแบบ Many-to-Many ระหว่าง Reports และ LINE Groups
- กำหนดตารางเวลาการส่งรายงานแบบรายวัน รายสัปดาห์ รายเดือน และรูปแบบที่เหมาะสมกับงาน
- สร้าง Scheduler/Execution Engine ที่ทำงานอัตโนมัติโดยไม่ต้องสร้าง Cron Job แยกตามรายงาน
- รองรับ Flex Message และ Message Type อื่นตามความเหมาะสม
- บันทึกประวัติการประมวลผลและผลการส่งรายงานเพื่อการตรวจสอบย้อนหลัง
- ลดการแก้ไข Source Code และลดภาระการบริหาร Cron Job เมื่อมีการเพิ่ม/เปลี่ยนรายงาน
- รองรับการขยายระบบและการเชื่อมต่อกับฐานข้อมูล/ระบบสารสนเทศหลายแหล่ง

3. Business Value และประโยชน์ที่คาดว่าจะได้รับ

ระบบนี้ไม่ได้เป็นเพียงระบบส่งข้อความ LINE แต่เป็น Reporting Distribution Platform

ที่เปลี่ยนการส่งรายงานจากงานเชิงเทคนิคแบบตายตัว ให้เป็นกระบวนการบริหารที่สามารถกำหนด ตรวจสอบ

และควบคุมได้จากส่วนกลาง

มิติ Business Value	ปัญหาเดิม	คุณค่าหลังพัฒนา	ตัวชี้วัด/ผลลัพธ์ที่คาดหวัง
ลดภาระงาน IT	เพิ่มรายงานต้องแก้ PHP/สร้าง Cron	เพิ่ม/แก้ Schedule และ Target ผ่าน Back Office	ลดงานแก้ Source Code สำหรับ Configuration
Operational Efficiency	การตั้งเวลาและกลุ่มกระจายอยู่หลายไฟล์	Configuration อยู่ในระบบกลาง	บริหารจากหน้าจอเดียว
Reliability	ตรวจสอบการส่งต้องดู Server/Cron	มี Execution และ Delivery Log	ตรวจสอบสถานะการส่งย้อนหลังได้ 100% ของงานที่ระบบประมวลผล
Scalability	จำนวน Cron เพิ่มตามจำนวนรายงาน	Scheduler กลางรองรับหลาย Report/Schedule	เพิ่มรายงานโดยไม่เพิ่ม Cron Job รายตัว
Management Visibility	รายงานกระจายตาม Group แต่ไม่มีภาพรวม	Dashboard แสดงงานที่ส่ง สำเร็จ/ล้มเหลว	เห็นสถานะรายงานประจำวันได้จาก Dashboard
Governance	การเปลี่ยนแปลงทำผ่าน Source Code	มีสิทธิ์ผู้ใช้และ Audit Trail	ตรวจสอบผู้สร้าง/แก้ไข/เปิดปิด Schedule ได้
Business Continuity	Cron/Script ผูกกับเครื่องหรือไฟล์	มี Retry และ Error Handling	ลดความเสี่ยงจากงานส่งล้มเหลว
Reuse	Flex/Query ผูกกับ Script	แยก Report Definition, Data และ Template	นำ Report เดิมไปส่งหลาย Group ได้

3.1 มูลค่าทางเศรษฐศาสตร์ที่สามารถประเมินได้

เพื่อให้หน่วยงานสามารถคำนวณ ROI จริงหลังติดตั้งระบบ ให้เก็บข้อมูล Baseline ก่อนและหลังใช้งาน เช่น จำนวนรายงาน จำนวนครั้งที่ต้องแก้ไข Cron/Source Code เวลาเฉลี่ยของผู้ดูแลระบบต่อการเพิ่มรายงาน และจำนวนครั้งที่การส่งรายงานล้มเหลว

ตัวแปรสำหรับประเมิน	สูตรตัวอย่าง	ข้อมูลที่หน่วยงานควรเก็บ
มูลค่าชั่วโมงงานที่ลดลง	$(\text{ชั่วโมงที่ลดลง/เดือน}) \times \text{ค่าแรงต่อชั่วโมง} \times 12$	เวลาจริงก่อนและหลังใช้ระบบ
Cost Avoidance จากการแก้ไขระบบ	$(\text{จำนวน Change ที่ลดลง}) \times \text{เวลาต่อ Change} \times \text{ค่าแรงต่อชั่วโมง}$	จำนวน Change รายงาน/ปี
มูลค่าจากลดความผิดพลาด	$(\text{จำนวน Incident ที่ลดลง}) \times \text{ค่าใช้จ่ายเฉลี่ยต่อ Incident}$	ประวัติ Incident
มูลค่าจากการขยายระบบ	จำนวนรายงานใหม่ที่เพิ่มได้โดยไม่เพิ่มงานพัฒนา	จำนวน Report/ปี

หมายเหตุด้าน Business Case

ตัวเลข ROI/ผลประโยชน์จริงไม่ควรสมมติใน TOR ให้หน่วยงานเก็บ Baseline จากระบบเดิมแล้วคำนวณผลจริงหลังใช้งาน 3-6 เดือน เพื่อใช้เป็น KPI ของโครงการ

4. ขอบเขตของระบบ

หัวข้อ	Module	ขอบเขต
4.1	LINE Group Management	จัดการรายชื่อกลุ่ม LINE, groupId, ชื่อกลุ่ม, สถานะ, คำอธิบาย และการตรวจสอบว่า OA ยังอยู่ในกลุ่ม
4.2	Report Management	สร้าง/แก้ไข/เปิดใช้งาน/ปิดใช้งาน Report Definition และกำหนด Data Source, Handler/Query และ Template
4.3	Target Management	กำหนดว่ารายงานแต่ละรายการจะส่งไปยัง Group ใดได้หลายกลุ่ม
4.4	Schedule Management	กำหนดเวลา ความถี่ วัน เวลา Timezone สถานะ และช่วงเวลาที่มีผล
4.5	Report Engine	Query ข้อมูล ประมวลผล Business Logic สร้างข้อความ/ Flex Message และส่งไปยัง Target
4.6	Scheduler	ค้นหา Schedule ที่ถึงเวลาและสร้างงานประมวลผลโดยไม่ต้องสร้าง Cron Job รายงานละตัว
4.7	Webhook	รับ join/leave/message event ตามที่จำเป็น และจัดการ Group Registration
4.8	Logging & Monitoring	เก็บ Execution Log, Delivery Log, Error และข้อมูลสำหรับ Dashboard
4.9	Back Office	หน้าจอจัดการ Groups, Reports, Targets, Schedules, Logs และ User/Roles
4.10	Integration	เชื่อมต่อ LINE Messaging API และฐานข้อมูล/ระบบสารสนเทศที่ได้รับอนุญาต

5. แนวคิดและสถาปัตยกรรมระบบ

สถาปัตยกรรมที่เสนอเป็น Modular Architecture โดยแยก Presentation, API, Scheduler, Report Engine, Data Access และ LINE Integration ออกจากกัน

Layer	Component	หน้าที่
ผู้ดูแลระบบ	Back Office	จัดการ Group / Report / Target / Schedule / Logs
Web/API	Next.js / Node.js API	Authentication, Authorization, CRUD และ orchestration
Scheduler	Node.js Scheduler	ค้นหา Job ที่ถึงเวลาและสร้าง Execution
Report Engine	Node.js Service	Query → Transform → Template → Message
Data Layer	PostgreSQL	Configuration, Job, Log, Audit
Integration	LINE Messaging API	Webhook, Group Summary, Push Message, Flex Message
Data Sources	DB/API ของหน่วยงาน	ข้อมูลสำหรับสร้างรายงาน

5.1 Flow หลักของระบบ

- 1) ผู้ดูแลเพิ่ม/ลงทะเบียน LINE Group โดย Invite LINE OA เข้า Group
- 2) LINE Platform ส่ง Webhook join event → ระบบรับ groupId

- 3) ระบบเรียก Group Summary API → ตรวจสอบ groupName และข้อมูลประกอบ
- 4) ระบบบันทึก Group และ groupId ลง PostgreSQL
- 5) ผู้ดูแลสร้าง Report Definition
- 6) ผู้ดูแลกำหนด Target Group ให้รายงาน
- 7) ผู้ดูแลกำหนด Schedule
- 8) Scheduler ตรวจสอบ Schedule ที่ถึงเวลา
- 9) Report Engine Query ข้อมูลและสร้าง Message/Flex Message
- 10) ระบบส่ง Push Message ไปยังแต่ละ groupId
- 11) ระบบบันทึกผลสำเร็จ/ล้มเหลวแยกราย Target

ข้อกำหนดเชิงสถาปัตยกรรม

ห้ามออกแบบให้รายงานแต่ละตัวต้องมี Cron Job ของตนเองเป็นหลัก ระบบต้องมี Scheduler กลาง และต้องสามารถเพิ่ม Report/Target/Schedule ผ่านฐานข้อมูลและ Back Office ได้โดยไม่ต้องสร้าง Cron Job ใหม่

6. Functional Requirements

ID	Feature	Requirement	Priority
FR-01	LINE Group Registration	รับ join event และบันทึก groupId โดยอัตโนมัติ	สูง
FR-02	Group Summary	เรียกข้อมูล groupName/pictureUrl จาก LINE API เมื่อจำเป็น	สูง
FR-03	Group Status	รองรับ Active/Inactive/Left และตรวจสอบสถานะ	สูง
FR-04	Report Definition	สร้าง/แก้ไข/ลบ/เปิดปิดรายงาน	สูง
FR-05	Report Handler	แยก Business Logic/Query จาก Scheduler	สูง
FR-06	Flex Template	รองรับ Flex JSON Template และตัวแปรจากผล Query	สูง
FR-07	Report Target	กำหนดหลาย Group ต่อ 1 Report	สูง
FR-08	Schedule	Daily/Weekly/Monthly/Custom และ Timezone Asia/Bangkok	สูง
FR-09	Scheduler	ค้นหาและประมวลผล Job ที่ถึงเวลา	สูง
FR-10	Retry	Retry งานที่ส่งไม่สำเร็จตาม Policy	กลาง-สูง
FR-11	Execution Log	บันทึกการเริ่ม/จบ/สถานะ/ข้อผิดพลาด	สูง
FR-12	Delivery Log	บันทึกผลการส่งแยกราย Group	สูง
FR-13	Manual Run	ผู้มีสิทธิ์สามารถ Run Now เพื่อทดสอบรายงาน	กลาง-สูง
FR-14	Test Send	ทดสอบส่งไปยัง Group ที่เลือกก่อนเปิด Schedule	สูง
FR-15	Dashboard	แสดงงานวันนี้และสถานะ Success/Failed	กลาง
FR-16	Audit Trail	บันทึกผู้สร้าง/แก้ไข/เปิดปิด/Run	สูง
FR-17	Leave Event	เมื่อ OA ออกจาก Group ให้เปลี่ยนสถานะและหยุดการส่ง	สูง

6.1 การจัดการ LINE Group

- ระบบต้องรองรับการลงทะเบียน Group โดยรับ groupId จาก Webhook join event
- ระบบควรเรียก GET /v2/bot/group/{groupId}/summary เพื่อดึง groupName และ pictureUrl ตามความเหมาะสม
- ต้องป้องกัน groupId ซ้ำในฐานข้อมูล
- ผู้ดูแลสามารถกำหนดชื่อเรียกภายในระบบ (Display Name) และคำอธิบายเพิ่มเติมได้ โดยไม่แก้ไข groupId ที่มาจาก LINE
- เมื่อได้รับ leave event ให้หยุดการส่งไปยัง Group นั้นโดยอัตโนมัติหรือเข้าสู่สถานะ Inactive ตาม Policy
- ต้องมีปุ่ม Test Send สำหรับทดสอบข้อความก่อนเปิดใช้งาน

6.2 การจัดการรายงาน

- Report Code ต้องไม่ซ้ำ และใช้เป็นตัวระบุรายงานสำหรับระบบ
- Report ต้องมี Name, Description, Handler, Template, Status และข้อมูลประกอบที่จำเป็น
- Business Logic ของรายงานต้องไม่ถูกฝังอยู่ใน Scheduler
- สามารถกำหนด Parameter เช่น วันที่เริ่มต้น วันที่สิ้นสุด หน่วยงาน หรือเงื่อนไขเฉพาะได้
- รองรับรายงานที่สร้างจาก SQL/View/Stored Function/API ตาม Data Source ที่ได้รับอนุญาต
- รองรับการทดสอบ Preview ก่อนส่งจริง

6.3 การกำหนดกลุ่มเป้าหมาย

- 1 Report : N LINE Groups และ 1 LINE Group : N Reports
- ต้องแสดงรายชื่อ Target ที่ Active เท่านั้นในการส่งจริง
- เมื่อ Group ถูก Inactive/Left ระบบต้องไม่นำ Group นั้นไปส่ง
- ควรมี Target Priority/Order หากรายงานมีหลายปลายทางและต้องการควบคุมลำดับ

6.4 การกำหนดเวลา

ตัวอย่าง	เวลา	Schedule Type
ทุกวัน	09:00	Daily
ทุกวันจันทร์	10:00	Weekly / Monday
ทุกวันศุกร์	15:00	Weekly / Friday
ทุกวันที่ 1	08:30	Monthly / Day 1
วันสุดท้ายของเดือน	16:30	Monthly / Last Day
Custom	ตามที่กำหนด	Cron-like / Rule

7. Non-Functional Requirements

ID	หัวข้อ	ข้อกำหนด
NFR-01	Availability	ระบบ Back Office/API ควรมี Availability ไม่น้อยกว่า 99.5% ต่อเดือน ไม่รวม Planned Maintenance
NFR-02	Security	ใช้ HTTPS, Secret Management, RBAC และตรวจสอบ LINE Webhook Signature
NFR-03	Performance	งานส่งรายงานทั่วไปต้องเริ่มประมวลผลภายในเวลาที่กำหนดจาก Scheduler โดยไม่ทำให้ API หลักค้าง
NFR-04	Scalability	สามารถเพิ่ม Report, Group, Schedule และ Target ได้โดยไม่ต้องเพิ่ม Cron Job รายตัว
NFR-05	Reliability	มี Retry/Timeout/Idempotency เพื่อป้องกันการส่งซ้ำจาก Job เดียวกัน
NFR-06	Observability	มี Structured Log และค้นหาประวัติการทำงานย้อนหลังได้
NFR-07	Maintainability	แยก Module และ Service ตามหน้าที่ พร้อมเอกสารติดตั้ง/Configuration
NFR-08	Backup	มีการสำรอง PostgreSQL และสามารถกู้คืนข้อมูล Configuration/Log ตามนโยบายหน่วยงาน
NFR-09	Time Zone	ใช้ Asia/Bangkok เป็น Time Zone มาตรฐานของ Business Schedule
NFR-10	Auditability	เก็บ Audit Trail ของการเปลี่ยนแปลง Configuration ที่มีผลต่อการส่งรายงาน

8. โครงสร้างข้อมูลและฐานข้อมูล

ฐานข้อมูลหลักเสนอให้ใช้ PostgreSQL โดยแยก Configuration, Execution และ Audit ให้ชัดเจน

Table	Purpose	Key Fields
line_groups	เก็บ LINE Group และ groupId	id, group_id, group_name, display_name, picture_url, status, created_at, updated_at
reports	นิยามรายงาน	id, report_code, report_name, handler, template_id, status, description
report_parameters	Parameter ของรายงาน	id, report_id, param_name, data_type, default_value, required
report_targets	ความสัมพันธ์ Report ↔ Group	id, report_id, group_id, priority, status
report_schedules	ตารางเวลา	id, report_id, schedule_type, cron_expression/rule, timezone, next_run_at, last_run_at, status
report_executions	ผลการประมวลผลรายรอบ	id, report_id, schedule_id, started_at, completed_at, status, error_message, correlation_id
report_deliveries	ผลส่งแยก Group	id, execution_id, group_id, status, response_code, error_message, sent_at
flex_templates	Template ของ Flex Message	id, template_code, template_name, template_json, version, status
users	ผู้ใช้งาน Back Office	id, username, display_name, status
roles	บทบาท	id, role_code, role_name
audit_logs	ประวัติการเปลี่ยนแปลง	id, user_id, action, entity, entity_id, before_data, after_data, created_at

8.1 ความสัมพันธ์หลัก

Relationship	Cardinality	รายละเอียด
--------------	-------------	------------

Reports → Report Targets	1:N	รายงานหนึ่งรายการมีหลาย Target
Line Groups → Report Targets	1:N	Group หนึ่งรับหลายรายงาน
Reports → Schedules	1:N	รายงานหนึ่งมีหลายตารางเวลาได้
Reports → Executions	1:N	รายงานหนึ่งมีประวัติการประมวลผลหลายครั้ง
Executions → Deliveries	1:N	การประมวลผลหนึ่งรอบมีผลส่งหลาย Group
Reports → Flex Templates	N:1 / versioned	หลายรายงานอาจใช้ Template เดียวกันหรือแยก Template ตาม Version

9. การเชื่อมต่อ LINE Messaging API

หลักการจาก LINE Platform

LINE Official Account สามารถเข้าร่วม Group Chat ได้เมื่อเปิด Allow bot to join group chats ใน LINE Developers Console โดยในขณะเดียวกัน Group หนึ่งสามารถมี LINE Official Account ได้เพียงหนึ่ง OA; เมื่อ OA ถูกเชิญเข้ากลุ่มจะมี join event และ groupId อยู่ใน source ของ event และ Push Message สามารถระบุ groupId ใน to ได้

ID	Component	Requirement
LINE-01	Webhook	POST endpoint สำหรับรับ Webhook Event
LINE-02	Signature Verification	ตรวจสอบ x-line-signature ก่อนประมวลผล
LINE-03	Join Event	รับ event.type = join และบันทึก groupId
LINE-04	Leave Event	รับ event.type = leave และเปลี่ยนสถานะ Group
LINE-05	Group Summary	เรียก /v2/bot/group/{groupId}/summary ตามความเหมาะสม
LINE-06	Push Message	POST /v2/bot/message/push โดย to = groupId
LINE-07	Flex Message	รองรับ message.type = flex และ template JSON
LINE-08	Error Handling	จัดการ HTTP error, timeout, retry และ log

9.1 เอกสารอ้างอิงทางเทคนิคของ LINE

Group chats and multi-person chats: <https://developers.line.biz/en/docs/messaging-api/group-chats>

Messaging API reference: <https://developers.line.biz/en/reference/messaging-api/nojs/>

Send Flex Messages: <https://developers.line.biz/en/docs/messaging-api/using-flex-messages/>

10. Scheduler และ Report Execution Engine

10.1 Scheduler

- ทำงานเป็น Service กลาง ไม่ผูกกับรายงานใดรายงานหนึ่ง
- ตรวจสอบ Schedule ที่ถึงเวลาเป็นรอบ ๆ เช่น ทุก 1 นาที หรือใช้ Queue/Scheduler ตามสถาปัตยกรรมที่ผู้พัฒนานำเสนอ
- สร้าง Execution Job พร้อม correlation_id เพื่อป้องกันการประมวลผลซ้ำ
- รองรับ Lock/Distributed Lock หากมีหลาย Instance
- รองรับการ Skip/Disable Schedule โดยไม่ต้องแก้ Source Code
- แสดง next_run_at และ last_run_at ใน Back Office

10.2 Report Execution

1. Load Report Definition
2. Resolve Parameters
3. Query/Call Data Source
4. Transform Data
5. Render Message/Flex Template
6. Resolve Active Target Groups
7. Send to each Group
8. Write Delivery Log
9. Update Execution Status

10.3 Retry และ Idempotency

- กำหนด Retry Policy เช่น 3 ครั้งสำหรับ Error ชั่วคราว โดยใช้ Exponential Backoff
- ไม่ Retry สำหรับ Error ที่เกิดจาก Configuration ผิดหรือ Target ถูกปิด/ไม่พบ
- ใช้ execution_id + group_id หรือ idempotency key เป็นตัวช่วยป้องกันการส่งซ้ำ
- เมื่อมีการส่งบาง Group สำเร็จและบาง Group ล้มเหลว ต้องบันทึกสถานะแยกราย Group

11. Back Office และสิทธิ์ผู้ใช้งาน

หน้าจอ	หน้าที่
Dashboard	งานวันนี้, Success/Failed, จำนวน Group, Report และ Schedule
LINE Groups	เพิ่ม/แก้ไข/ดูสถานะ/ค้นหา/ทดสอบส่ง
Reports	สร้าง/แก้ไข/Preview/Run Now/Enable/Disable
Targets	กำหนด Group เป้าหมายของแต่ละ Report
Schedules	ตั้งเวลา/แก้ไข/Enable/Disable/ดู Next Run
Execution Logs	ดูประวัติการประมวลผล
Delivery Logs	ดูผลการส่งแยกราย Group
Templates	จัดการ Flex Template และ Version
Users & Roles	จัดการผู้ใช้งานและสิทธิ์
Audit Logs	ตรวจสอบการเปลี่ยนแปลง Configuration

11.1 Roles ที่เสนอ

Role	สิทธิ์
Super Admin	ทุกฟังก์ชัน, User/Roles, Configuration
Report Admin	Reports, Templates, Targets, Schedules
Operator	ดู Logs, Run Now ตามสิทธิ์, Test Send
Viewer	ดู Dashboard/Report/Logs แบบอ่านอย่างเดียว

12. ความมั่นคงปลอดภัยและการคุ้มครองข้อมูล

- ใช้ HTTPS สำหรับ Webhook และ Back Office/API
- LINE Channel Access Token และ Channel Secret ต้องเก็บใน Secret/Environment Variable ห้าม Hard-code ใน Source Code
- ตรวจสอบ LINE Webhook Signature ทุก Request ก่อนประมวลผล
- ใช้ RBAC สำหรับ Back Office
- กำหนด Session/Token Expiration และ Password Policy ตามมาตรฐานหน่วยงาน
- ใช้ Parameterized Query/ORM/Stored Function เพื่อป้องกัน SQL Injection
- จำกัดการเข้าถึงฐานข้อมูลตาม Principle of Least Privilege
- ไม่บันทึก Token, Secret หรือข้อมูลส่วนบุคคลที่ไม่จำเป็นลง Log
- กำหนด Retention ของ Execution/Delivery/Audit Log ตามนโยบายหน่วยงาน
- กรณีรายงานมีข้อมูลส่วนบุคคล ต้องกำหนด Data Classification และ Masking/Minimization ตามความเหมาะสม

13. Logging, Monitoring และ Audit Trail

Log Type	ข้อมูลขั้นต่ำ	วัตถุประสงค์
Application Log	timestamp, level, service, message, correlation_id	ตรวจสอบระบบ
Execution Log	report, schedule, start/end, status, error	ตรวจสอบการทำงานรายงาน
Delivery Log	execution, group, response code, status, sent_at	ตรวจสอบการส่งราย Group
Audit Log	user, action, entity, before/after	ตรวจสอบการเปลี่ยน Configuration
Webhook Log	event type, groupId masked, timestamp, result	Troubleshooting Webhook

Monitoring ที่แนะนำ

ควรมี Alert เมื่อรายงานสำคัญล้มเหลว, Scheduler ไม่ทำงาน, Queue ค้างเกิน threshold, LINE API error สูงผิดปกติ หรือไม่มี Execution เกิดขึ้นตามช่วงเวลาควรมี

14. API และการเชื่อมต่อกับระบบอื่น

Method	Endpoint ตัวอย่าง	วัตถุประสงค์
POST	/api/webhooks/line	รับ LINE Webhook
GET	/api/line-groups	รายการ Group
POST	/api/line-groups/test-send	ทดสอบส่งข้อความ
GET	/api/reports	รายการ Report
POST	/api/reports	สร้าง Report
POST	/api/reports/{id}/run	Run Now
GET	/api/reports/{id}/targets	รายการ Target
POST	/api/reports/{id}/targets	เพิ่ม Target
GET	/api/reports/{id}/schedules	รายการ Schedule
POST	/api/reports/{id}/schedules	สร้าง Schedule
GET	/api/executions	Execution Logs
GET	/api/deliveries	Delivery Logs

15. การย้ายจากระบบ PHP/Cron เดิม

แนวทางที่แนะนำ: ไม่ควร Rewrite ทั้งหมดในครั้งเดียว

ให้ใช้ Migration แบบ Incremental โดยเริ่มจาก LINE Group/Webhook และ Scheduler จากนั้นเชื่อม Scheduler ไปเรียก Report เดิม แล้วจึงทยอยย้าย Report แต่ละตัวเข้าสู่ Report Engine ใหม่

Phase	งาน	รายละเอียด	ผลลัพธ์
Phase 1	LINE Group + Webhook	สร้าง Group Registry และรับ join/leave event	ระบบเดิมยังทำงาน
Phase 2	Central Scheduler	Scheduler เรียก Script/PHP เดิม	ลด Cron รายตัว
Phase 3	Report Engine	ย้าย Query/Business Logic/Flex ที่ละ Report	ทำ Parallel Run ได้
Phase 4	Back Office	เปิดให้จัดการ Report/Target/Schedule	ลดการแก้ Code
Phase 5	Decommission	ยกเลิก Cron/PHP ที่ย้ายเสร็จและเก็บ Archive	เหลือระบบกลาง

15.1 หลักเกณฑ์ย้ายรายงาน

- รายงานเดิมต้องมีผลลัพธ์เทียบเท่าระบบเดิมก่อน Cutover
- ทดสอบ Query, จำนวนข้อมูล, การคำนวณ และ Flex Rendering
- ช่วง Parallel Run ให้ส่งไป Test Group หรือเปรียบเทียบ Output โดยไม่ส่งซ้ำให้ผู้ใช้จริง
- มี Rollback Plan สำหรับแต่ละ Report
- ย้ายสำเร็จแล้วจึงปิด Cron เดิม

16. ผลส่งมอบ (Deliverables)

ID	Deliverable	รายละเอียด
D01	Source Code	Source Code ของ Frontend/Backend/Scheduler/Integration พร้อม Repository
D02	Database	Schema, Migration, Seed/Reference Data และ ERD
D03	Back Office	ระบบบริหาร Groups, Reports, Targets, Schedules, Logs
D04	LINE Integration	Webhook + Messaging API Integration
D05	Report Engine	Engine สำหรับ Query/Transform/Template/Delivery
D06	Scheduler	Scheduler/Queue/Job Processing
D07	Security	Authentication/RBAC/Signature Verification/Secret Configuration
D08	Documentation	Architecture, API, Database, Deployment และ Admin Manual
D09	Test	Test Case, Test Result และ UAT Evidence
D10	Migration	แผนและผลการย้ายรายงานเดิม
D11	Deployment	Deployment Script/Docker Compose หรือวิธีติดตั้งตามมาตรฐานหน่วยงาน
D12	Training	อบรมผู้ดูแลระบบและผู้ใช้งานที่เกี่ยวข้อง

17. แผนดำเนินงานและ Timeline

ช่วงลำดับ	กิจกรรม	ผลส่งมอบ
1-2	เก็บ Requirement / วิเคราะห์ระบบเดิม	BRD/Requirement Baseline
3-4	ออกแบบ Architecture / Database / UI	Design Approval
5-7	พัฒนา LINE Webhook + Group Management	Group Registry
6-9	พัฒนา Report/Schedule/Target	Core Back Office
8-11	พัฒนา Scheduler + Report Engine + Logs	Execution Platform
10-12	พัฒนา Flex Template / Integration	Report Delivery
11-13	ย้าย/พัฒนารายงานชุดแรก	Pilot Reports
13-15	System Integration Test / Security Test	SIT
15-16	UAT / Training / Documentation	UAT Sign-off
17	Production Deployment / Cutover	Go-Live

หมายเหตุ

Timeline 120 วันเป็นกรอบตัวอย่างสำหรับ TOR ควรปรับตามจำนวนรายงานเดิม จำนวน Data Source ความพร้อมของ Server/Network/LINE OA และกระบวนการจัดซื้อจัดจ้างของหน่วยงาน

18. เกณฑ์การทดสอบและตรวจรับ

ID	Test Area	Acceptance Criteria	ผล
AT-01	LINE Group	Invite OA เข้า Group → ได้ join event → groupId ถูกบันทึก	ผ่าน/ไม่ผ่าน
AT-02	Group Summary	ชื่อ Group ถูกดึงและแสดงในระบบ	ผ่าน/ไม่ผ่าน
AT-03	Report	สร้าง Report และ Preview/Run Now ได้	ผ่าน/ไม่ผ่าน
AT-04	Target	1 Report ส่งได้หลาย Group และ 1 Group รับหลาย Report	ผ่าน/ไม่ผ่าน
AT-05	Schedule	กำหนด Daily/Weekly/Monthly และตรวจ next run ได้	ผ่าน/ไม่ผ่าน
AT-06	Delivery	Flex Message ถึง Group เป้าหมายครบและถูกต้อง	ผ่าน/ไม่ผ่าน
AT-07	Failure	จำลอง LINE/API/DB failure และตรวจ Retry/Log	ผ่าน/ไม่ผ่าน
AT-08	Idempotency	Job เดียวกันไม่สร้างการส่งซ้ำจากการ Retry	ผ่าน/ไม่ผ่าน
AT-09	Leave	OA ออกจาก Group → ระบบหยุดการส่งไป Group	ผ่าน/ไม่ผ่าน
AT-10	Audit	การแก้ไข Report/Target/Schedule มี Audit Log	ผ่าน/ไม่ผ่าน
AT-11	Security	Webhook signature invalid ต้องถูกปฏิเสธ	ผ่าน/ไม่ผ่าน
AT-12	Migration	รายงานที่ย้ายจากระบบเดิมให้ผลเทียบเท่าหรือได้รับการรับรองจากเจ้าของข้อมูล	ผ่าน/ไม่ผ่าน

19. การรับประกันและบำรุงรักษา

- รับประกันระบบไม่น้อยกว่า 12 เดือนนับจากวันที่ตรวจรับงวดสุดท้าย หรือระยะเวลาตามที่กำหนดในสัญญา
- แก้ไข Defect ที่ทำให้ระบบใช้งานไม่ได้โดยไม่คิดค่าใช้จ่ายเพิ่มเติมภายในระยะเวลาประกัน
- กำหนด Severity และ SLA ในการตอบรับ/แก้ไข เช่น Critical, High, Medium, Low
- ให้คำแนะนำและสนับสนุนการแก้ไข Configuration/Deployment ที่เกี่ยวข้องกับระบบ
- การเปลี่ยนแปลง Feature ใหม่ที่อยู่นอก Scope ให้ดำเนินการตาม Change Request

20. ข้อกำหนดด้านผู้พัฒนา/ผู้รับจ้าง

- มีประสบการณ์พัฒนา Web Application และ Backend API
- มีประสบการณ์ Node.js/TypeScript หรือเทคโนโลยีเทียบเท่า
- มีประสบการณ์ PostgreSQL หรือ RDBMS ที่เทียบเท่า
- มีประสบการณ์ REST API, Webhook และ Third-party API Integration
- มีความรู้ด้าน LINE Messaging API และการจัดการ LINE Official Account หรือสามารถแสดงแผนงานรองรับได้
- มีความรู้ด้าน Docker/Linux/Nginx หรือ Infrastructure ที่หน่วยงานใช้งาน
- มีความรู้ด้าน Security, Authentication, RBAC และ Secret Management
- สามารถส่งมอบ Source Code, Documentation และ Deployment Configuration ได้ครบถ้วน

21. เงื่อนไขสำคัญและข้อพึงระวัง

- ระบบต้องไม่พึ่งพา Cron Job แยก Report เป็นสถาปัตยกรรมหลัก
- ห้ามเก็บ LINE Channel Access Token/Channel Secret ใน Source Code หรือ Repository
- การส่งไป Group ต้องใช้ groupId ที่ได้รับจาก LINE Platform และ OA ต้องเป็นสมาชิกของ Group
- Group หนึ่งสามารถมี LINE Official Account ได้เพียงหนึ่ง OA ตามข้อกำหนดของ LINE
- ต้องมีการตรวจสอบ Webhook Signature
- ไม่ควรให้ผู้ใช้งานทั่วไปสามารถเขียน SQL อิสระบนฐานข้อมูล Production โดยตรง
- ควรใช้ View/Stored Function/Service Layer เป็น Data Contract สำหรับรายงานที่มีความสำคัญ
- ต้องแยก Execution Log กับ Delivery Log เพื่อแยกปัญหา “รายงานประมวลผลไม่ได้” ออกจาก “ส่งบาง Group ไม่ได้”
- ต้องมี Timezone ที่ชัดเจน โดยกำหนด Asia/Bangkok เป็นค่าเริ่มต้น
- ต้องออกแบบให้สามารถเปลี่ยน LINE OA/Channel ในอนาคตได้โดยไม่ผูกมัดระบบทั้งหมดกับ Credential เดียว

22. สรุปภาพรวมโครงการ

10. LINE Auto Reports จะทำหน้าที่เป็นระบบกลางสำหรับบริหารการส่งรายงานไปยัง LINE Group โดยมี LINE Official Account เป็นช่องทางการกระจายข้อมูล
11. หัวใจของระบบคือการแยก Configuration ออกจาก Source Code: Group, Report, Target และ Schedule ถูกบริหารผ่านฐานข้อมูลและ Back Office

12. Scheduler กลางทำหน้าที่สร้างงานตามเวลา ส่วน Report Engine ทำหน้าที่ Query → Transform → Generate Message/Flex → Deliver → Log
13. การออกแบบดังกล่าวช่วยให้สามารถเพิ่มรายงานใหม่ เปลี่ยนกลุ่มเป้าหมาย หรือเปลี่ยนเวลาส่งได้โดยไม่ต้องแก้ PHP/Cron ทุกครั้ง
14. ระบบสามารถเริ่มต้นจากระบบ PHP/Cron เดิมและทยอย Migration แบบ Incremental เพื่อลดความเสี่ยงต่อระบบที่ใช้งานจริง

Business Outcome ที่ต้องการ

จาก “ระบบ Script ที่ส่งรายงานตาม Cron” → เป็น “Platform สำหรับบริหารการกระจายรายงานแบบอัตโนมัติ” ที่มี Configuration Management, Scheduling, Multi-Target Delivery, Monitoring, Audit และสามารถขยายจำนวนรายงาน/กลุ่มได้โดยไม่ต้องเพิ่มความซับซ้อนของ Cron Job ตามจำนวนรายงาน

ภาคผนวก A: ตัวอย่างการใช้งานจริง

กลุ่ม	รายงาน	เวลา	Target
กลุ่มผู้บริหาร	รายงานสรุปผู้บริหารประจำวัน	09:00 ทุกวัน	ผู้บริหาร
เจ้าหน้าที่ IT	System Health / Incident	08:30 ทุกวัน	IT
คณะศิลปศาสตร์	สรุปนักศึกษา/กิจกรรม	10:00 ทุกวันจันทร์	คณะศิลปศาสตร์
งานทะเบียน	นักศึกษาใหม่	10:00 ทุกวัน	งานทะเบียน
ผู้บริหาร + IT	รายงานโครงการ Digital Platform	15:00 ทุกวันศุกร์	ผู้บริหาร + IT

ภาคผนวก B: ตัวอย่าง Flex Message Data Contract

ตัวอย่างแนวคิด: Report Query ดึงข้อมูลเชิงโครงสร้าง แล้ว Template นำข้อมูลไปประกอบเป็น Flex Message โดยไม่ให้ Scheduler รู้รายละเอียดของ Layout

```
{
  "reportCode": "DAILY_STUDENT_SERVICE",
  "data": {
    "date": "2026-08-14",
    "total": 1245,
    "growth": 12.5
  }
}
```

→ Flex Template
 → Generate Flex JSON
 → Push Message
 → LINE Group

ภาคผนวก C: ตัวอย่างโครงสร้าง Repository

```

line-auto-reports/
├── apps/
│   ├── web/          # Back Office
│   └── api/          # REST API
├── services/
│   ├── scheduler/    # Scheduler / Job Runner
│   ├── report-engine/ # Query / Transform / Render
│   └── line-service/  # LINE API / Webhook
├── database/
│   ├── migrations/
│   ├── seeds/
│   └── views/
├── reports/
│   ├── daily-student-service/
│   ├── system-health/
│   └── finance/
├── templates/
│   └── flex/
├── docs/
└── docker-compose.yml
  
```

เอกสารอ้างอิงหลัก: [LINE Developers – Group chats](#)

[LINE Developers – Messaging API reference](#)

[LINE Developers – Send Flex Messages](#)